# Chapter 8: Data Flow Analysis

**Download this chapter as a PDF for offline reading:**
Chapter-08-Data-Flow-Analysis.pdf

Pattern matching on the AST is useful, but many security and quality questions are really about *values*: does attacker-controlled data reach this sensitive operation? Data-flow analysis answers that question by tracking how values move through the program.

## Local and Global Flow

CodeQL provides two complementary forms of data flow.

### Local data flow

Local flow tracks values *within a single function*. It is precise and inexpensive.

```
DataFlow::localFlow(source, sink)
```

Example: find values that flow locally into a `console.log` call (see `examples/chapter08/local-flow.ql`):

```
/**
 * @name Local flow into console.log
 * @description Finds values that flow locally into console.log.
 * @kind problem
 * @id mustlearn/js-local-flow
 * @tags security
 * @problem.severity recommendation
 * @precision high
 */
import javascript

from DataFlow::Node source, DataFlow::Node sink
where
  DataFlow::localFlow(source, sink) and
  sink = any(CallExpr c | c.getCalleeName() = "console.log").getArgument(0).flow()
select sink, "Value flows locally into console.log."
```

Use local flow whenever the interesting behavior is confined to one function.

### Global data flow

Global flow tracks values *across function boundaries*. It is more powerful and more expensive. It is expressed with a *configuration* (covered in the next section).

| Kind | Scope | Precision | Cost |
|---|---|---|---|
| Local | Single function | Higher | Lower |
| Global | Whole program | Lower | Higher |

Prefer local flow when it is sufficient.

## Configurations, Additional Steps, and Barriers

### Configurations

A data-flow configuration is a class that extends `DataFlow::Configuration` (or `TaintTracking::Configuration`). You override predicates to define the sources, sinks, and optional barriers or additional flow steps.

Skeleton (see `examples/chapter08/global-flow-skeleton.ql`):

```
class MyConfig extends DataFlow::Configuration {
  MyConfig() { this = "MyConfig" }

  override predicate isSource(DataFlow::Node source) {
    // Define sources
  }

  override predicate isSink(DataFlow::Node sink) {
    // Define sinks
  }
}
```

You then query the configuration:

```
from MyConfig cfg, DataFlow::PathNode source, DataFlow::PathNode sink
where cfg.hasFlowPath(source, sink)
select sink.getNode(), source, sink, "message"
```

### Additional steps

Sometimes the default modeling is incomplete. You can add extra flow steps with `isAdditionalFlowStep`:

```
override predicate isAdditionalFlowStep(DataFlow::Node pred, DataFlow::Node succ) {
  // Custom step: e.g. through a particular library call
}
```

### Barriers

A barrier stops flow. Common uses include sanitizers or validation functions:

```
override predicate isBarrier(DataFlow::Node node) {
  node = any(CallExpr c | c.getCalleeName() = "sanitize").getArgument(0).flow()
}
```

(See `examples/chapter08/barrier-example.ql`.)

Barriers are evaluated before additional steps, so a value that hits a barrier does not continue.

## Exercises

1. Modify `local-flow.ql` so it only reports flow that originates from a function parameter.

2. Turn the global-flow skeleton into a real query that tracks remote sources to `eval`.
3. Add a barrier that stops flow through calls to a function named `escapeHtml`.
4. Write a query that uses only local flow to find assignments of string literals to a variable named `secret`.

Solutions and additional examples live in the companion repository under `examples/chapter08/`.

## Next Steps

You can now track pure data flow both locally and globally, and you know how to shape that flow with configurations, additional steps, and barriers. The next chapter introduces taint tracking (flow that is *not* value-preserving) and full path queries.

All examples referenced above live in the companion repository:
github.com/rod-trent/MustLearnCodeQL
under `examples/chapter08/`.

*Must Learn CodeQL* — Rod Trent
Companion repository: github.com/rod-trent/MustLearnCodeQL